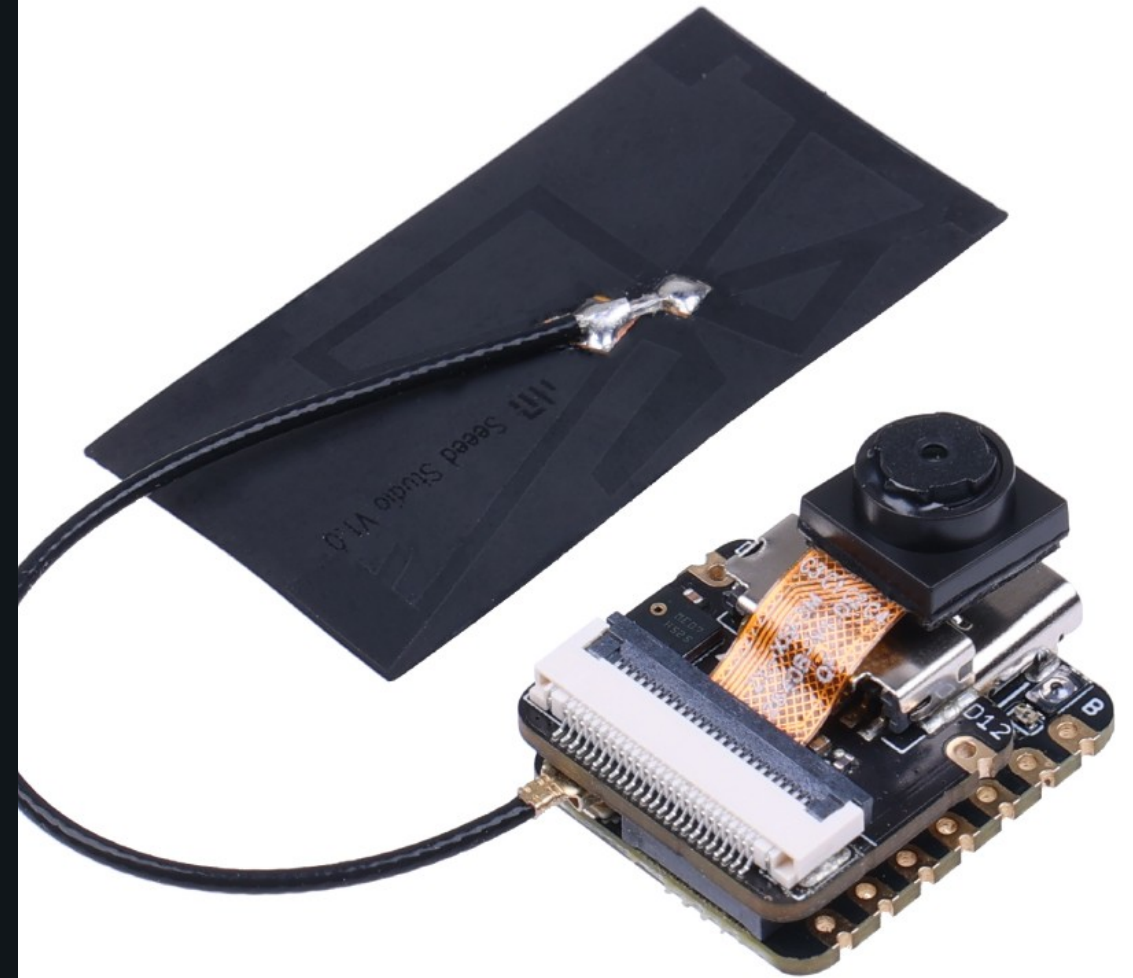


Programming responsive hardware

Microcontrollers, GPIO, and nonblocking timing



Microcontrollers



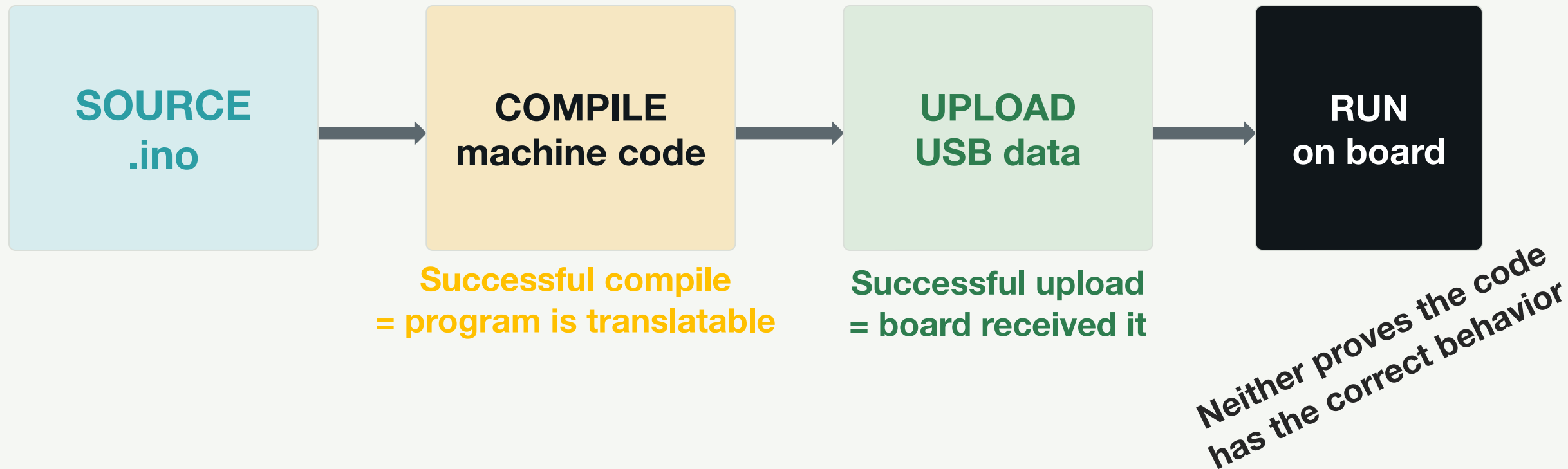
— processor executes instructions

— memory holds the uploaded program

— pins connect computation to hardware

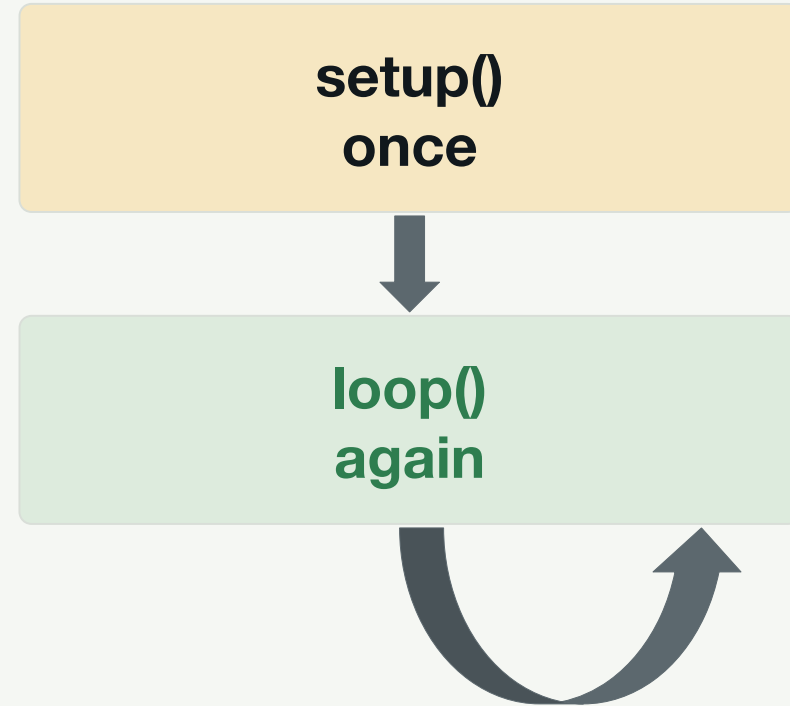
Unlike a laptop, the board is built to live inside another device.

Compiling and Uploading



setup() and loop()

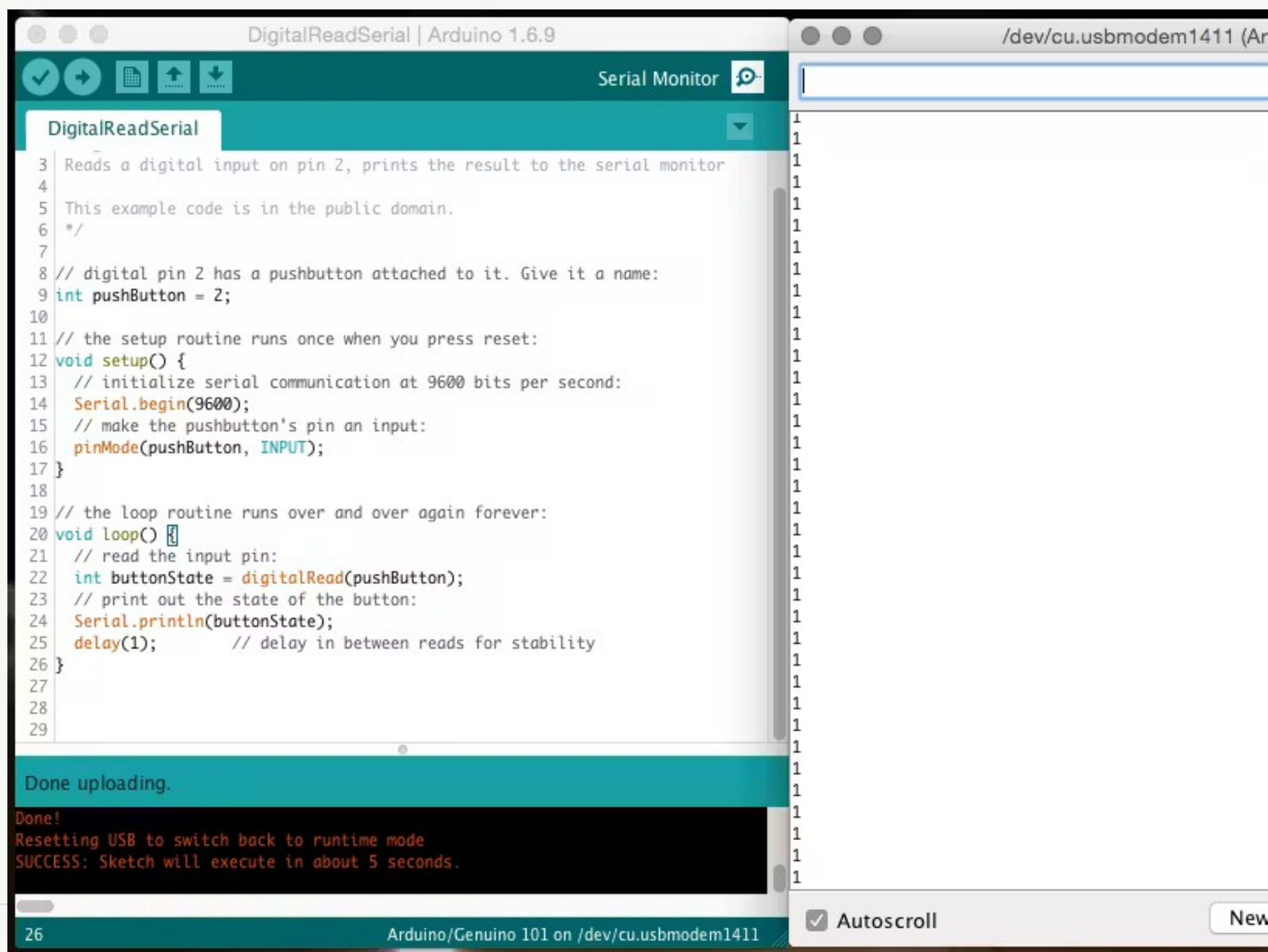
```
void setup() {  
  Serial.begin(115200);  
}  
  
void loop() {  
  Serial.println("again");  
}
```



Serial Output

Serial output is useful for debugging

Note: set baud rate to the same value in both the program and serial monitor. 9600 works well



DigitalReadSerial | Arduino 1.6.9

Serial Monitor

DigitalReadSerial

```
3 Reads a digital input on pin 2, prints the result to the serial monitor
4
5 This example code is in the public domain.
6 */
7
8 // digital pin 2 has a pushbutton attached to it. Give it a name:
9 int pushButton = 2;
10
11 // the setup routine runs once when you press reset:
12 void setup() {
13   // initialize serial communication at 9600 bits per second:
14   Serial.begin(9600);
15   // make the pushbutton's pin an input:
16   pinMode(pushButton, INPUT);
17 }
18
19 // the loop routine runs over and over again forever:
20 void loop() {
21   // read the input pin:
22   int buttonState = digitalRead(pushButton);
23   // print out the state of the button:
24   Serial.println(buttonState);
25   delay(1); // delay in between reads for stability
26 }
27
28
29
```

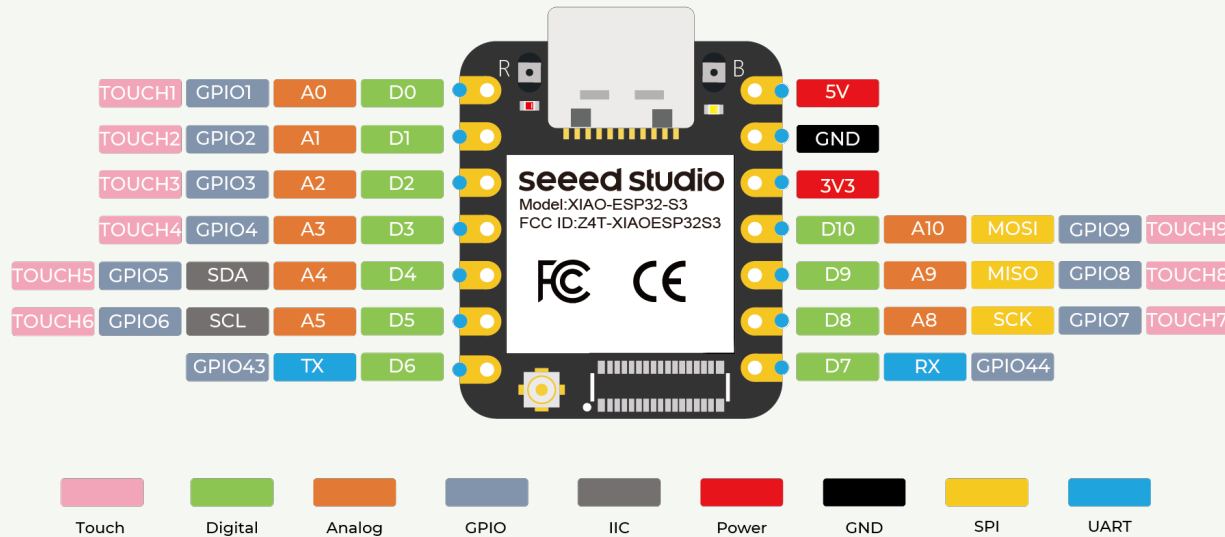
Done uploading.

Done!
Resetting USB to switch back to runtime mode
SUCCESS: Sketch will execute in about 5 seconds.

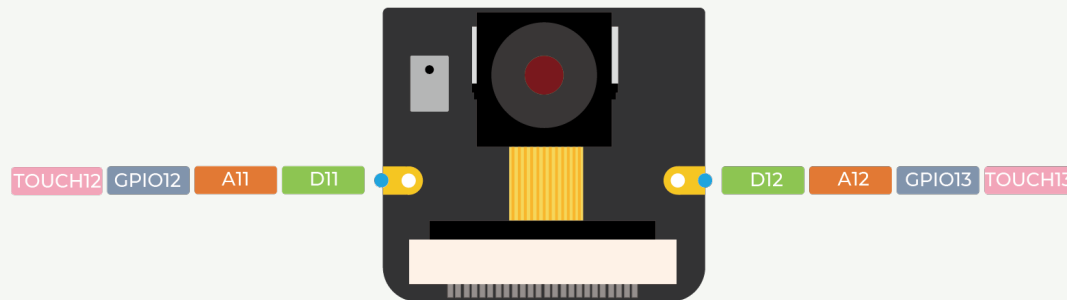
26 Arduino/Genuino 101 on /dev/cu.usbmodem1411

Autoscroll

General-Purpose Input/Output (GPIO)



```
const int LED_PIN = 3;  
  
pinMode(LED_PIN,  
OUTPUT);
```

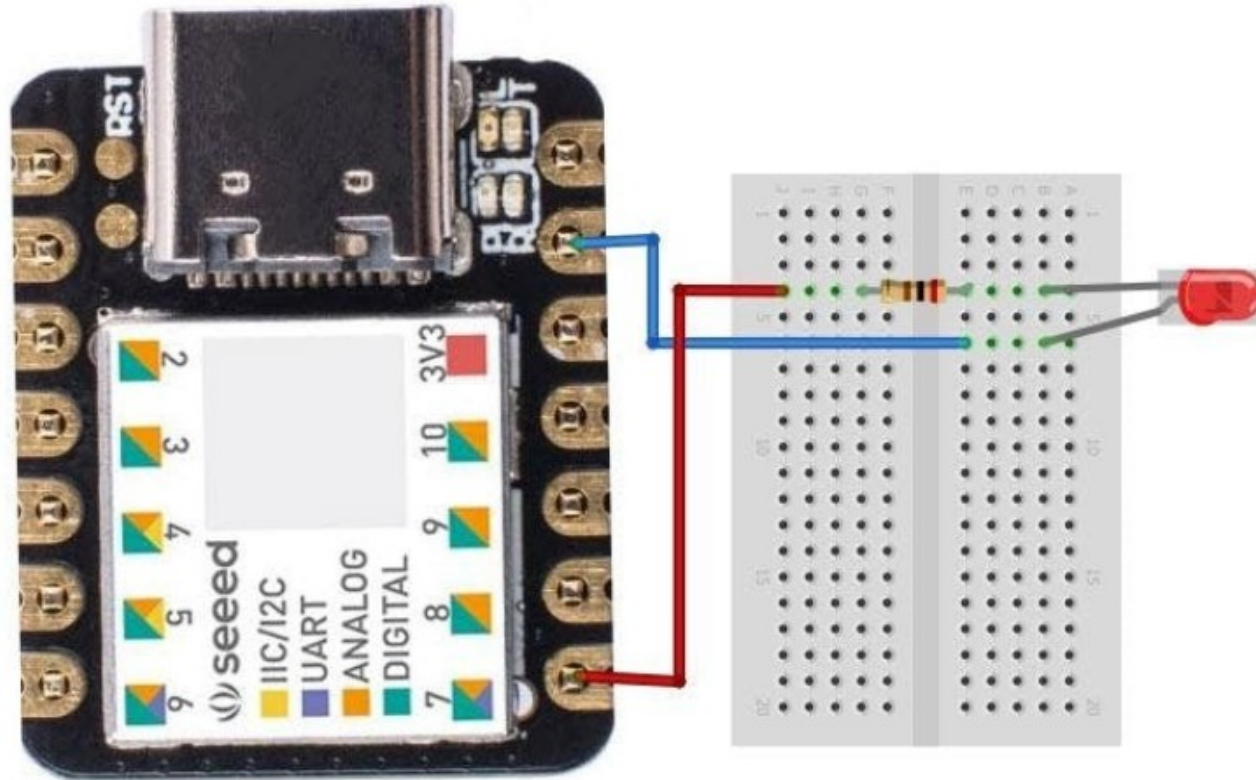


software name (LED_PIN)

silk-screen label (D3)

electrical pad

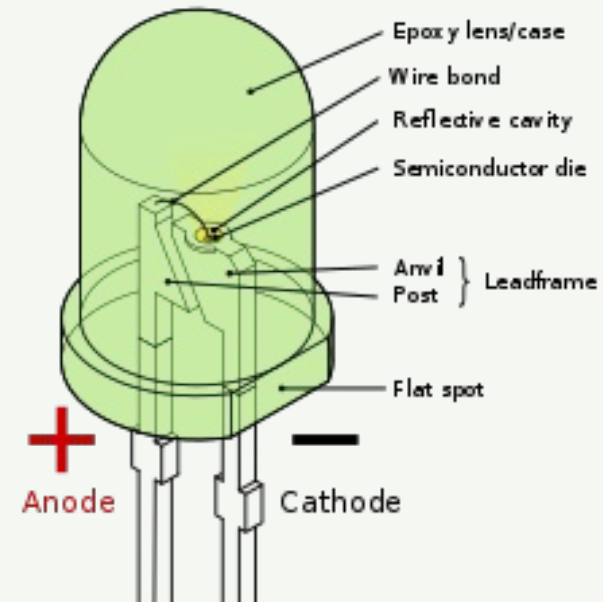
LED Output Circuit



Concepts in this circuit

- 1 LED polarity
- 2 Current-limiting resistor
- 3 Breadboard connectivity
- 4 Digital output pin

LED Polarity



— long leg: anode (+)

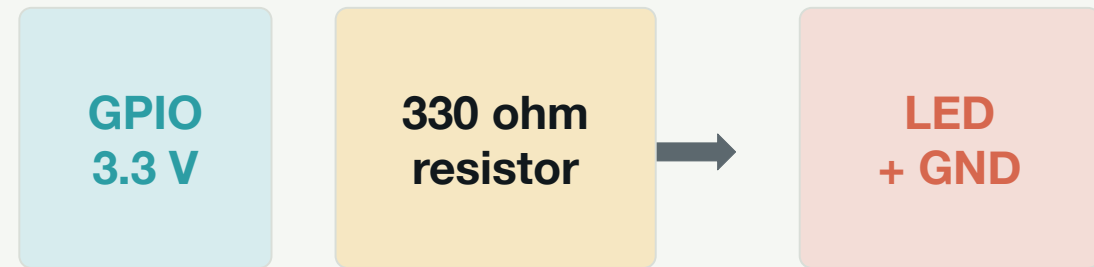
— short leg + flat edge: cathode (-)

Reversing the LED usually produces no light because the intended current path is blocked.

Current-Limiting Resistors

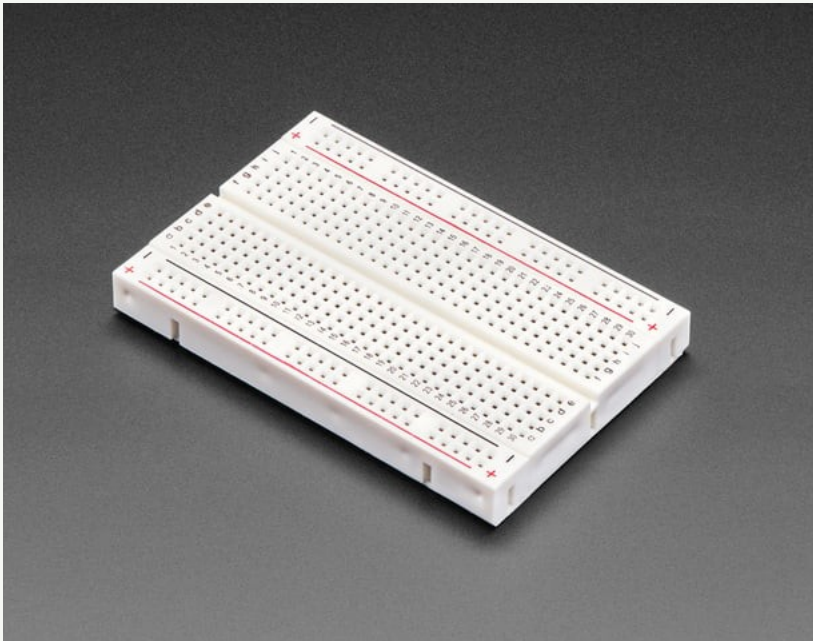
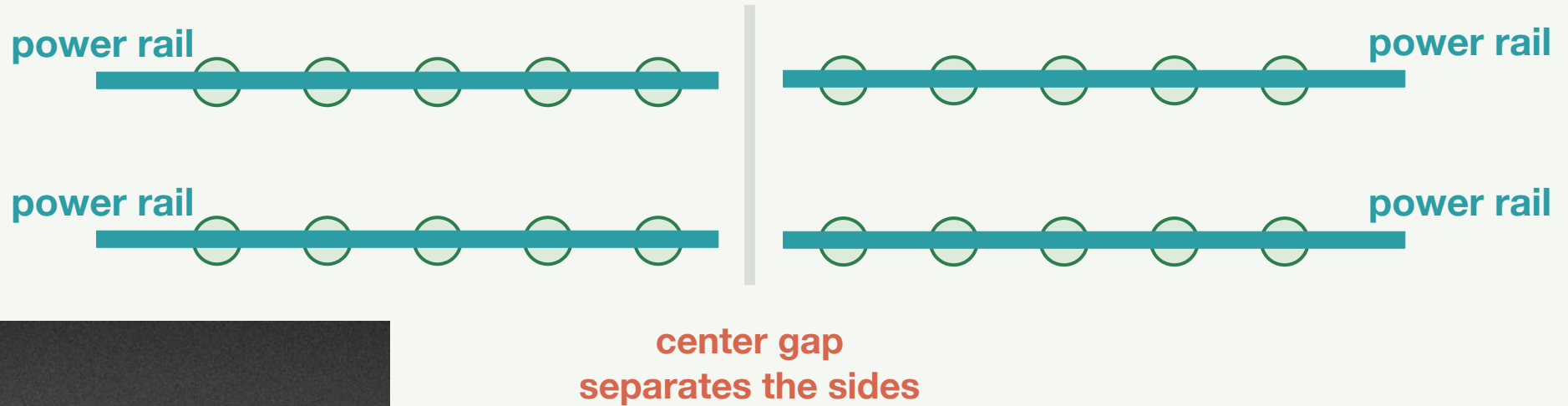


orange orange brown gold



The resistor is part of the circuit's safety,
not a software option.

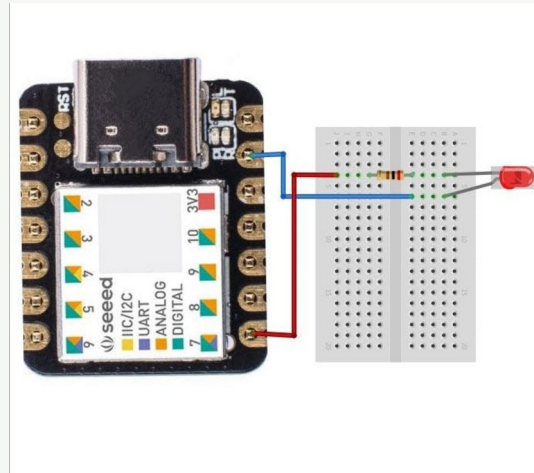
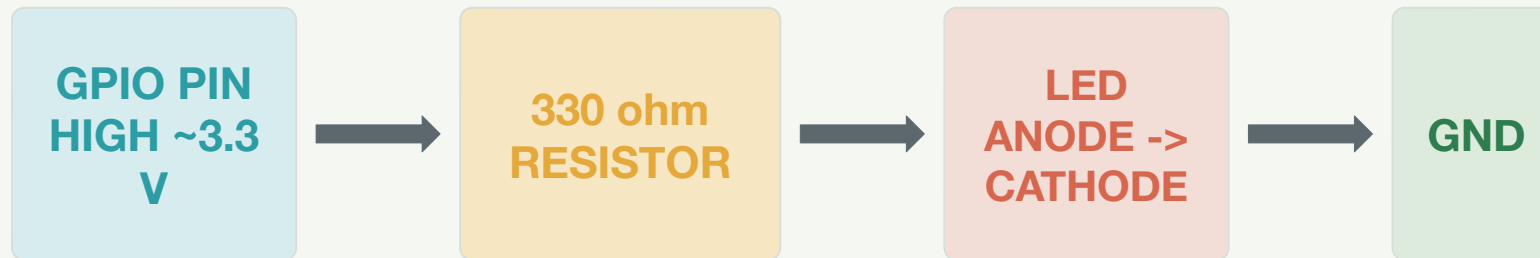
Breadboard Connectivity



A wire and a component lead is electrically connected only when they share in a connected group.

Building the LED Circuit

current can flow only when every connection exists



- 1 the selected GPIO pin changes voltage
- 2 current passes through the resistor
- 3 current enters the LED's anode
- 4 ground completes the return path

Configuring and Driving an Output

```
void setup() {  
  pinMode(LED_PIN, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(LED_PIN, HIGH);  
}
```



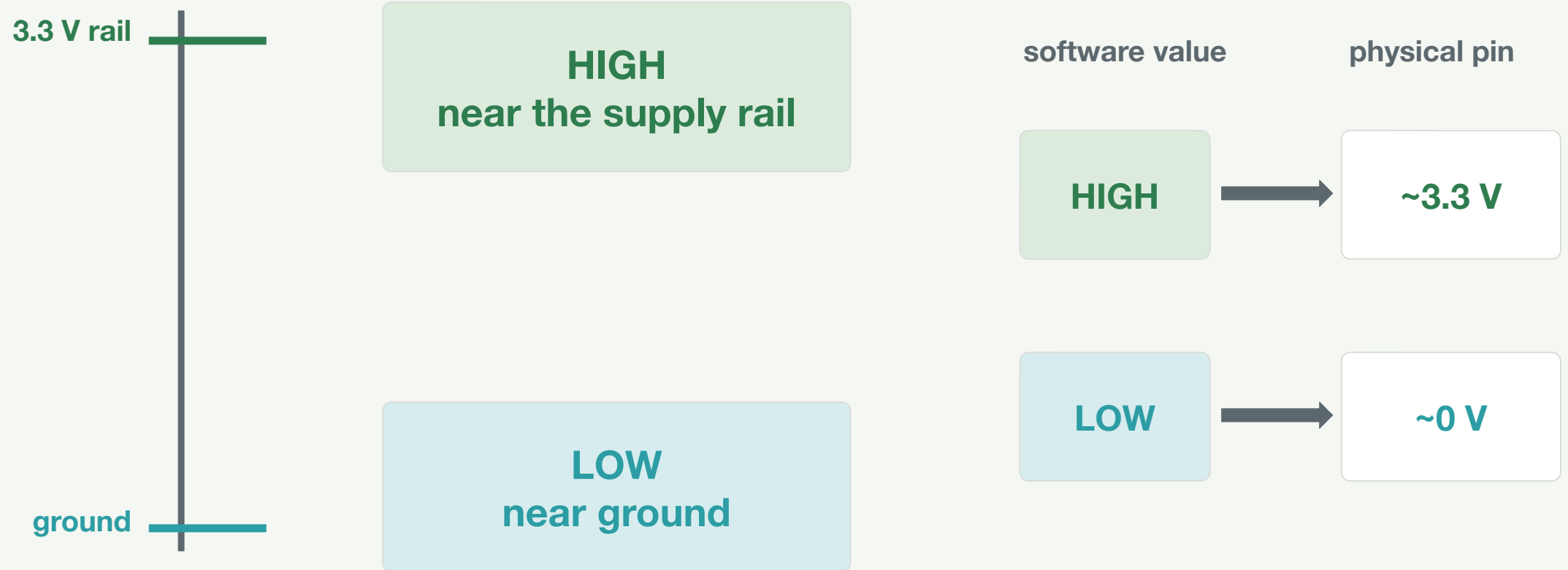
OUTPUT
the pin drives the circuit



HIGH
the output moves toward 3.3 V

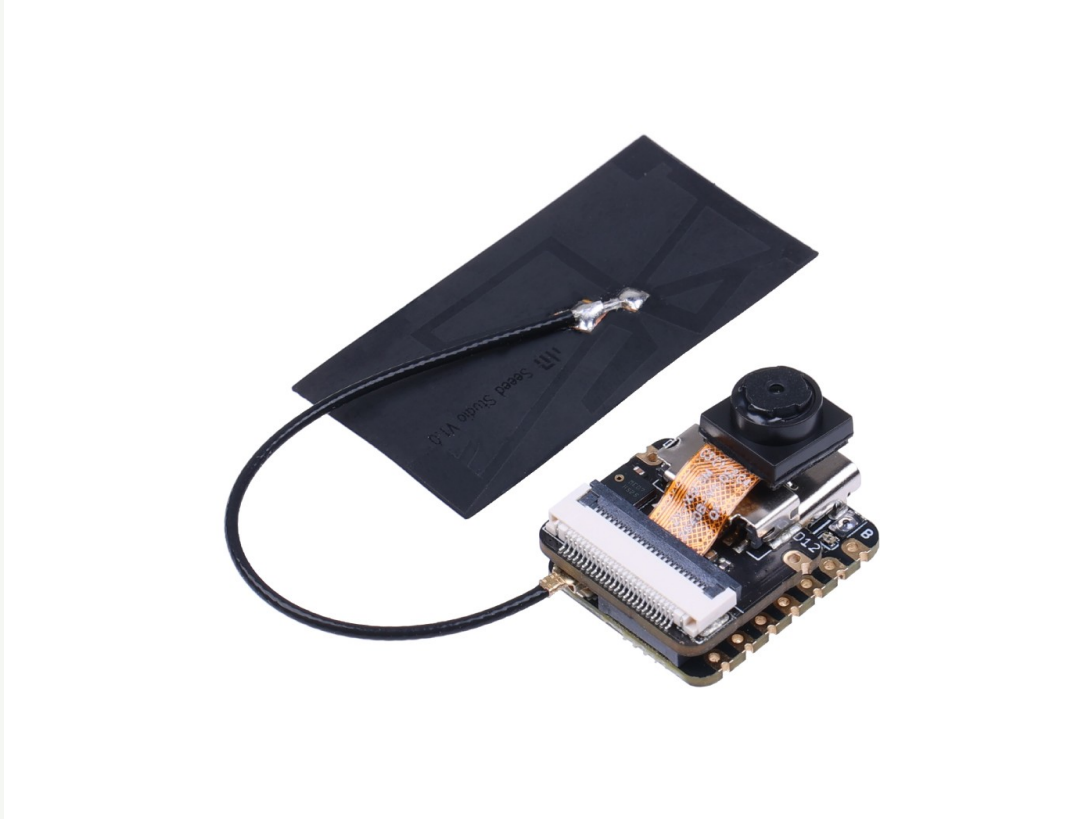
Correct pin + correct role + correct level + correct circuit

GPIO Voltage Levels



M02 will examine the voltage ranges that digital inputs interpret as 0 and 1.

Active-Low Outputs



```
digitalWrite(LED_BUILTIN, LOW)
```



LED ON

HIGH -> LED OFF

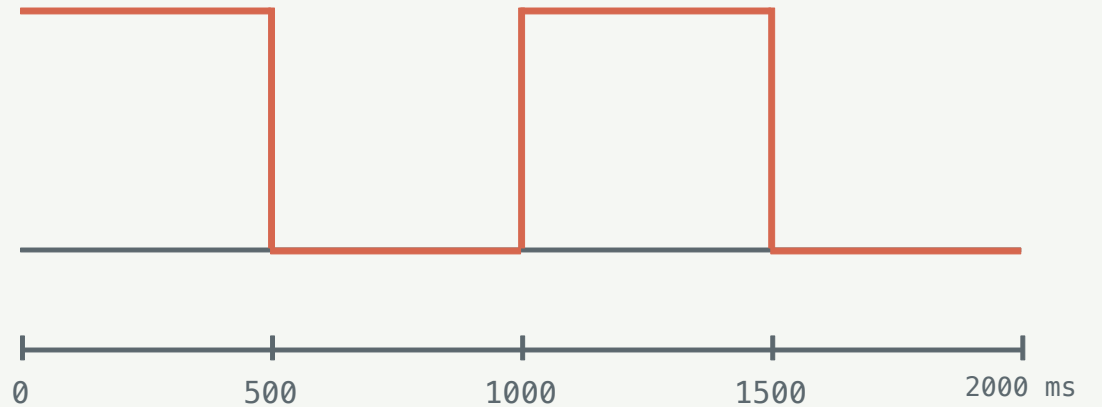
The LED is connected from 3.3 V to the pin: LOW sinks current and turns it on.

Timing

Blocking Timing

```
digitalWrite(RED_LED, HIGH);  
delay(500);  
digitalWrite(RED_LED, LOW);  
delay(500);
```

RED_LED



For one job, waiting and timing look like the same thing.

Multiple Timed Behaviors

RED



BLUE



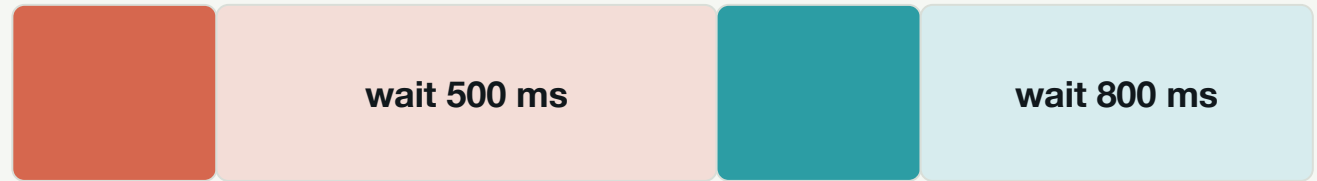
Both behaviors must progress during the same `loop()`.

How would you write the first attempt?

Blocking Delays

```
toggle(RED_LED);  
delay(500);  
  
toggle(BLUE_LED);  
delay(800);
```

actual loop timeline



next RED change: 1300 ms later

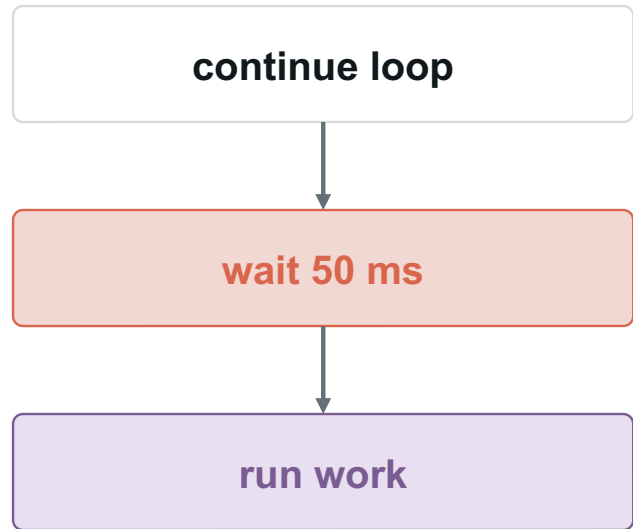
The program cannot service another job while
`delay()` is running.

This is blocking timing.

Blocking and Nonblocking Timing

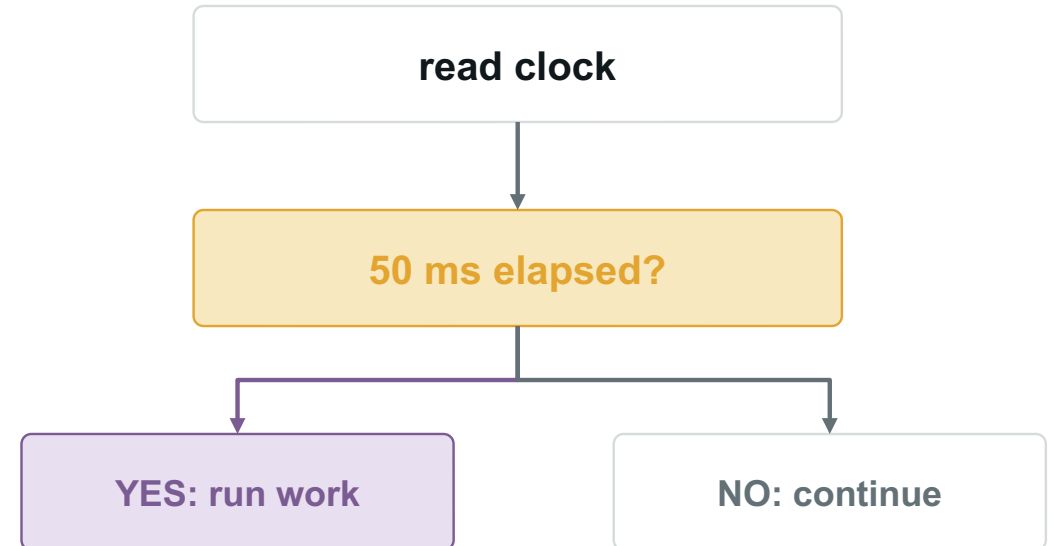
Task: run this work every 50 ms

Blocking with delay()



Main execution pauses during the wait.

Nonblocking with a clock check

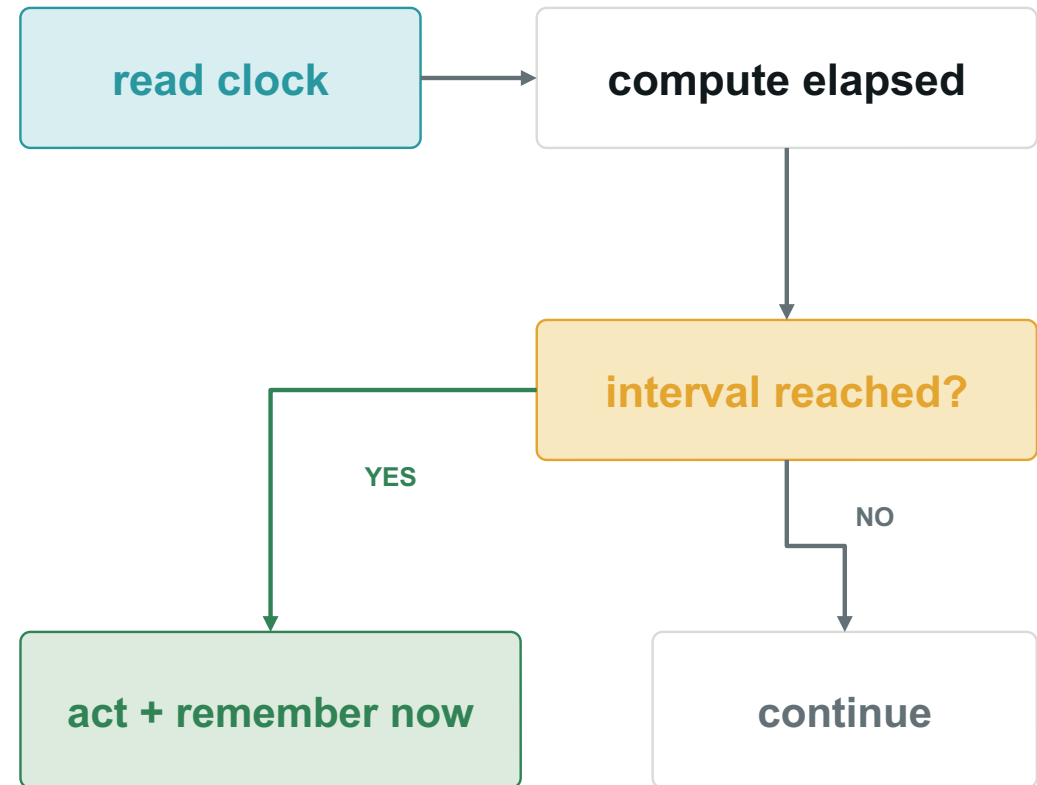


The loop keeps returning to every responsibility.

Clock-Based Timing

```
unsigned long now = millis();  
  
if (now - redLast >= 500) {  
    redLast = now;  
    redOn = !redOn;  
}
```

elapsed = now - last



Reusable Periodic Service

```
bool serviceDue(uint32_t nowMs,
                uint32_t intervalMs,
                uint32_t *lastRunMs) {
    uint32_t elapsedMs = nowMs - *lastRunMs;
    if (elapsedMs < intervalMs) return false;

    *lastRunMs = nowMs;
    return true;
}
```



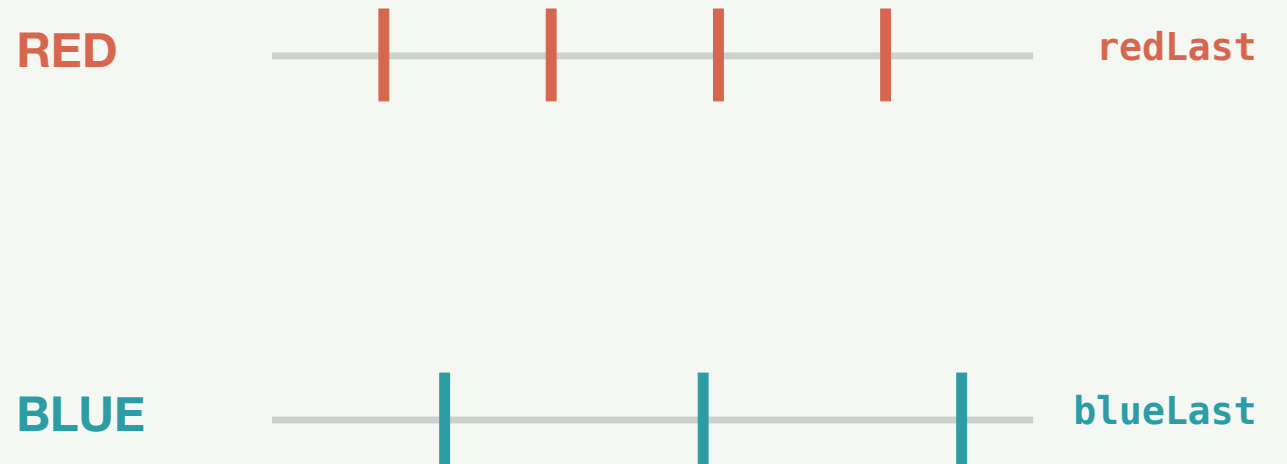
```
if (serviceDue(nowMs, 50,
               &lastSensorMs)) {
    sampleSensor();
}
```

- 1 The caller owns the remembered timestamp.
- 2 &lastSensorMs lets the helper update it.
- 3 False continues the loop; true runs the service.

One helper can schedule many independent services because each call supplies its own timestamp.

Independent Timers

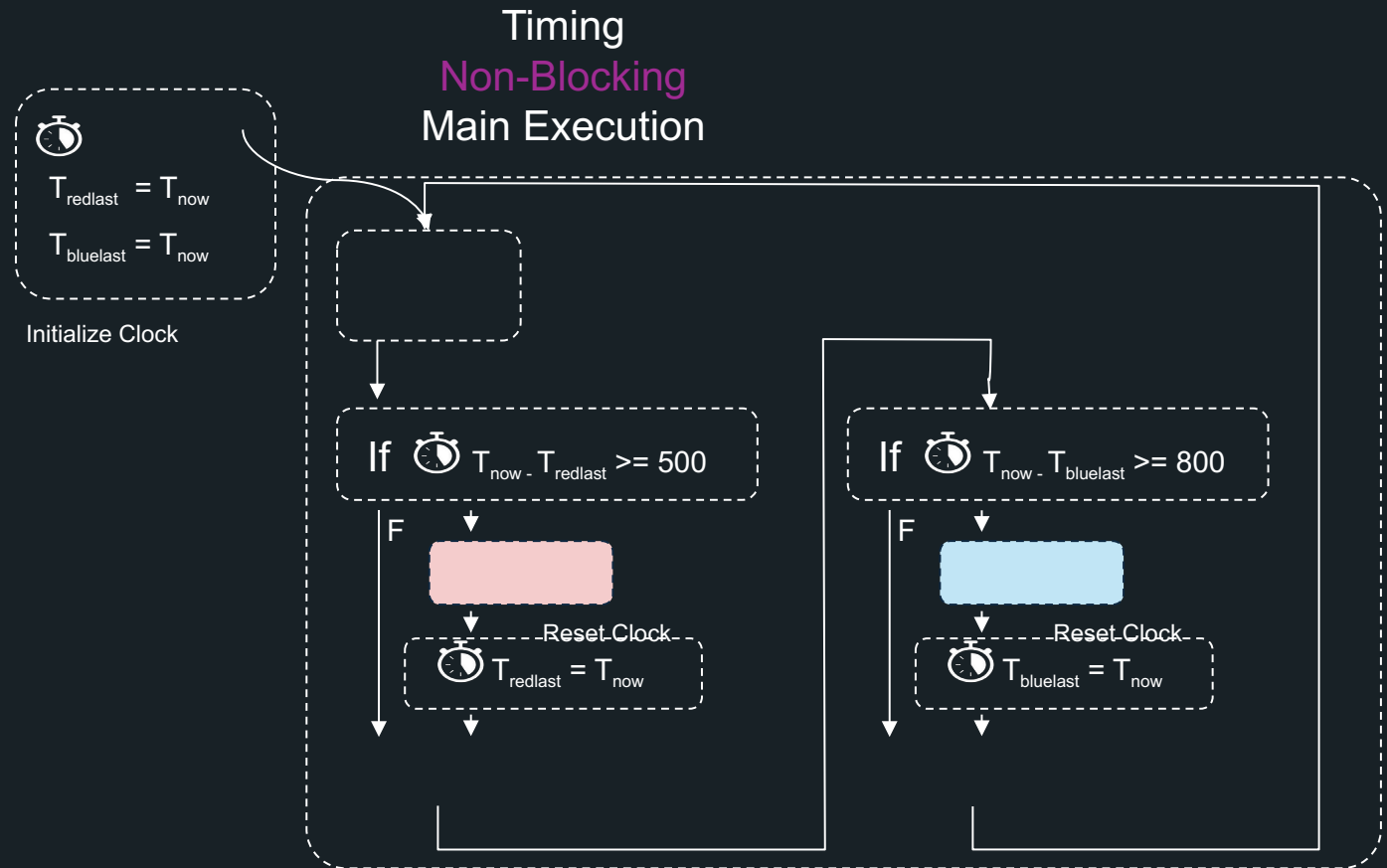
```
if (now - redLast >= 500) {  
    redLast = now;  
    toggleRed();  
}  
  
if (now - blueLast >= 800) {  
    blueLast = now;  
    toggleBlue();  
}
```



No delay ties the two schedules together.

Independent Timer Flow

```
if (now - redLast >= 500) {  
    redLast = now;  
    toggleRed();  
}  
  
if (now - blueLast >= 800) {  
    blueLast = now;  
    toggleBlue();  
}
```



Polling

now	red elapsed	blue elapsed	work due	result
100 ms	100	100	none	continue
500 ms	500	500	red	toggle red
800 ms	300	800	blue	toggle blue
1000 ms	500	200	red	toggle red
1600 ms	600	800	red + blue	toggle both

Polling means the loop checks repeatedly; work happens only when a condition is true.